

John Zachary Fitch

Independent AI researcher and systems engineer

zack@internetuniverse.org | github.com/johnzfitch | ORCID [0009-0007-7953-1531](https://orcid.org/0009-0007-7953-1531)

I'm a mathematician who builds things. Half my work is research on geometry-aware sequence models, memory, optimization, and what's going on inside transformers. The other half is the tooling that researchers and AI agents run on, mostly in Rust and Python. When something is slow or broken, I measure it and get a reproduction before I try to fix it.

Research

I run the Transporter program, where I study the geometry inside transformers and build model architectures from the exceptional Jordan algebra (the 27-dimensional Albert algebra). I take constructions from algebra and geometry, turn them into models I can actually run, and test them. The questions I keep coming back to: what does a model hold on to, how does it change as it learns, and can its structure show us better ways to learn?

- I built the Albert Engine (private repo), a tested numerical runtime for the Albert algebra. I use it to prototype sequence models, memory mechanisms, optimizers, and diagnostics that work with that geometry.
- I study transformer representations and how optimizers behave numerically, and I run causal interventions on pretrained open models to see which geometric structure they actually depend on.
- I test ideas with numerical checks and head-to-head comparisons under controlled conditions.
- I write down my predictions before an experiment, mark each claim as proved, measured, or conjectured, and report the limits and the negative results next to the positive ones.

Papers and code (public):

- **The Barnes–Gindikin Symbol at Fractional Rank: Continuation Without a Determinant Carrier, Positivity Without a Cone.** Paper source and exact-arithmetic verification code, archived on Zenodo: [DOI 10.5281/zenodo.21713316](https://doi.org/10.5281/zenodo.21713316) / [gindikin-rank](https://github.com/johnzfitch/gindikin-rank)
- **Reading the Residual.** Uses the geometry of symmetric cones to give an exact, computable error certificate for the inverse square roots that Kronecker-factored optimizers like Shampoo, SOAP, and K-FAC compute in low precision. [kl-shampoo-gindikin-bridge](https://github.com/johnzfitch/kl-shampoo-gindikin-bridge)
- **Communication transport experiments.** Complete outputs, with null and control runs, from transport experiments on Pythia-70M, Pythia-160M, and GPT-2. [communication-transport](https://github.com/johnzfitch/communication-transport)

More on the [research page](#).

Origin

The research question goes back to two pages of notes I wrote in a linear algebra class at SRJC in 2019. If two brains do the same things in different places, is there a map that sends each one onto shared functional pieces, where the difference becomes readable? Now I'm asking the same question about transformers.

OPENAI CODEX

Ghost in the Codex Machine

Codex release builds ran a hardening routine before `main()` that stripped `LD_*` and `DYLD_*` from the environment. Every tool subprocess inherited that environment, so CUDA and MKL couldn't find their libraries and fell back to slow paths without any error. I found the commit that turned it on, built a reproduction, and wrote it up in issue #8945. An OpenAI maintainer wrote the fix from my write-up, it shipped in rust-v0.80.0, and the release notes thank me by name.

PROOF

 [Issue #8945](#) - [Fix PR #8951](#)
 [rust-v0.80.0 release notes](#)

From my repro harness (numbers vary by setup)

- MKL/BLAS: ~2.71s -> ~0.239s (11.3x)
- CUDA: the libraries load again, so affected setups stop falling back to the CPU

Release notes excerpt:

"Special thanks to @johnzfitch for the detailed investigation and write-up in #8945."

Selected projects (public)

- **llmx** (Rust core, JS/WASM web; live at [llm.cat](#)): a codebase indexer that runs only on your machine. BM25 and neural embeddings (mdbr-leaf-ir on Burn) are merged with reciprocal rank fusion, and chunking is deterministic and content-hashed.
- **claude-warden** (Shell/OTEL, ★60+): security hooks and subagent budgets for Claude Code, with a built-in web viewer for the OTEL traces it records.
- **claude-cowork-linux** (Linux, ★400+): runs Claude Desktop natively on Linux. The host OS stands in for the Mac VM, the app runs inside bubblewrap, and the ASAR is handled from outside the sandbox instead of hooked from inside it. Cowork, Code, and Chat work; dispatch and channels are stripped out.
- **dota** (Rust): a post-quantum secrets manager. Hybrid ML-KEM-768 + X25519, Argon2id key derivation, SQLCipher at rest, and a YubiKey or SoloKey (HMAC-SHA1) to unlock it.

- **pyghidra-lite** (Python/MCP, ★35+): an MCP server for Ghidra that keeps token use low. It's in the official MCP registry as `io.github.johnzfitch/pyghidra-lite` (v0.1.1).
- **codex-xtreme** (Rust): builds optimized, patched Codex binaries (includes `codex-patcher`).
- **burn-plugin**: a Claude Code plugin and skills for the Burn deep learning framework.
- **Observatory** (WebGPU): spots AI-generated images right in the browser (live).
- **SpecHO v2** (Python): a 161-dimensional linguistic fingerprint for detecting AI-written text.
- **definitelynot.ai** (PHP/JS): a text sanitizer and API that catches Unicode tricks.
- **Iconics** (Python): an icon library for docs, 8k+ icons, searchable by meaning.

How I work

- I'd rather build from first principles than polish an existing design.
- I go back to ideas that got dropped or didn't work the first time and try them again with today's tools.
- I take working systems apart to find the assumptions nobody checked, then rebuild the parts that don't hold up.
- I'm most useful on hard problems where people still argue about the answer.
- On a new team I find my useful subspace early. I stay steady in a crisis and under a deadline.
- My workflow mixes math, AI-assisted coding, and experiments.

Core skills

Research

PyTorch, NumPy, transformer analysis, experiment design, numerical linear algebra, LaTeX

Rust

Systems where speed matters, CLI tools, data structures, code that has to be correct

Python

Tooling, analysis pipelines, automation, experiments I can rerun

Web

WebGPU and WASM apps, ML inference in the browser, apps that work offline

Systems

Linux, NixOS, DNS, automated TLS, containers, security hardening

Agent integration

MCP servers, skills and plugins, tool-driven workflows

Before this: TBD Liquids (2014–2023)

I started TBD Liquids in 2014 selling e-liquid I mixed by hand on Reddit's e-cigarette classifieds, and I ran it for nearly nine years. The mission was to help adults quit smoking cigarettes. The FDA regulated what we made as a tobacco product, and the work meant handling hazardous substances.

- Built the brand, The Best Damn Liquids (federally trademarked), on made-to-order mixing, a panel of about 50 outside flavor testers, and a price ceiling of \$11.99 per 30ml.
- Served roughly 40,000 customers by 2019 and filled more than 200,000 orders, with a team of up to six.
- Filed a Premarket Tobacco Product Application before FDA's September 2020 deadline, then spent years working through guidance that never committed to anything, FOIA requests included. FDA refused to file the application.
- Said no to opportunities that pulled us away from the mission, even profitable ones. We closed in July 2023, in our strongest revenue year.

What it taught me:

- How to keep going for years when a regulator won't give you a straight answer.
- How to run manufacturing, hiring, and fulfillment day to day, and how to handle a crisis.
- How to work under public scrutiny, for nearly nine years of it. I stand behind the work and I'm comfortable defending it on the record.

Infrastructure (self-hosted)

I run production services on my own bare-metal servers:

- NixOS, configured declaratively, so builds are reproducible and upgrades are atomic and easy to roll back.
- My own authoritative DNS, with wildcard certificates issued automatically over DNS-01 (RFC 2136).
- Post-quantum layers: hybrid SSH key exchange, and WireGuard with Rosenpass.

Education

UC Berkeley - B.A. in Mathematics, May 2022.

Santa Rosa Junior College - A.S. in Mathematics and an associate degree in Economics, August 2020.

What I'm looking for

I'm looking for researchers, labs, and engineering teams to work with on model architecture, learning algorithms, interpretability, and agent infrastructure, especially where a hard research question needs a working implementation and a clean experiment. I work best with people who measure their results, are clear about who owns what, and keep the bar high.